

ASP.NET Core :-

Q1)

1. What are the fundamental differences between the original AS.NET and ASP.NET Core?

ASP.NET:

- Runs only on Windows.
- Based on the .NET Framework.
- Runs mainly on IIS.
- Monolithic and heavier.
- Dependency Injection is not built-in.

ASP.NET Core:

- Cross-platform (Windows, Linux, macOS).
- Runs on modern .NET.
- Can run on Kestrel, IIS, Nginx, or Apache.
- Lightweight and modular.
- Built-in Dependency Injection.
- Higher performance.

2. What does it mean for an API to be 'Stateless'?

A stateless API means that each request from the client must contain all the information required to process the request.

The server does not store any session information between requests. Every request is independent.

3. Break down the anatomy of an HTTP Request URL.

Example URL:

`https://api.example.com/products/10?category=electronics`

Parts:

- Protocol: https
- Domain: api.example.com
- Path: /products/10 (resource location)
- Query Parameters: ?category=electronics (extra filters or parameters)

4. What are the primary HTTP Methods (Verbs) and their standard uses?

GET → Retrieve data

POST → Create new data

PUT → Update existing data completely

PATCH → Update part of existing data

DELETE → Remove data

5. What is the role of Program.cs in an ASP.NET Core application?

Program.cs is the entry point of the ASP.NET Core application.

It is responsible for:

- Creating the web application builder
- Registering services (Dependency Injection)
- Configuring middleware
- Starting and running the application

6. Why is Swagger/OpenAPI typically enabled only in the Development environment?

Swagger is usually enabled only in development because:

- It exposes all API endpoints.

- It could create security risks in production.
- It is mainly used for testing and documentation during development.

7. What is the core concept of Dependency Injection (DI)?

Dependency Injection is a design pattern where objects receive their dependencies from external sources rather than creating them themselves.

This improves flexibility, maintainability, and testability.

8. Explain the three Service Lifetimes in ASP.NET Core DI.

Transient: A new instance is created every time the service is requested.

Scoped: One instance is created per HTTP request.

Singleton: A single instance is created and shared across the entire application lifetime.

9. Why is it a Best Practice to register services against an Interface rather than a concrete Class?

Registering services with interfaces allows:

- Loose coupling
- Easier testing (mocking)
- Flexibility to change implementations
- Better maintainability

10. What are the Launch Profiles found in launchSettings.json?

Launch profiles define how the application runs during development.

They specify:

- Application URL

- Port numbers
- Environment variables
- How the project starts when debugging

11. How does JSON facilitate data exchange in APIs?

JSON (JavaScript Object Notation) is a lightweight data format used to exchange data between clients and servers.

Example:

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 1200  
}
```

Advantages:

- Easy to read and write
- Lightweight
- Language independent
- Fast for data exchange

Q2)

Scenario A : creates the dependency directly, which causes tight coupling and makes the code harder to test and maintain.

Scenario B : uses Dependency Injection through an interface, creating loose coupling, improving flexibility, testability, and maintainability. Therefore, Scenario B is the better design.

So B is best

Q3)

When a request is received, ASP.NET Core uses the **Dependency Injection container** to create the **ProductController**. The container resolves its dependencies by creating a **ProductService**, which in turn requires an **IRepository**. Since **IRepository** is registered as **SqlRepository**, the DI container injects a **SqlRepository** instance. The controller action then calls the service, which uses the repository to save the data.

Q4)

- **Transient:** Creates a new instance each time → `AreSameInstance = false`
- **Scoped:** One instance per request → `AreSameInstance = true`
- **Singleton:** One instance for the entire application → `AreSameInstance = true`

Q5)

- **Controller A** receives **MailgunEmailService** because ASP.NET Core injects the **last registered implementation** when a single service is requested.
- **Controller B** receives **three services** because `IEnumerable<IEmailService>` injects **all registered implementations** in registration order.